

```
In [ ]: import marimo as mo
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['lines.linewidth'] = 2
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3
plt.rcParams['font.size'] = 10
plt.rcParams['axes.labelsize'] = 11
plt.rcParams['axes.titlesize'] = 12
```

CL677: Assignment 3 - Euler-Maruyama Simulation of SDEs

Modelling Stochastic and Turbulent Transport (Spring 2025-26)

Students: Sarthak Mishra (22b0432) | Pratyush Ranjan (22b0326) **Instructor:** Prof. Jason Picardo

Introduction

We simulate two stochastic differential equations (SDEs) using the **Euler-Maruyama (EM)** scheme:

$$x_{n+1} = x_n + f(x_n) \Delta t + g(x_n) \sqrt{\Delta t} \xi_n, \quad \xi_n \sim \mathcal{N}(0, 1)$$

- **Question A:** Ornstein-Uhlenbeck process — $dx = -ax \, dt + dW$
 - **Question B:** Geometric random walk — Itô and Stratonovich interpretations of $dx = x \, dW$
-

Question A: Ornstein-Uhlenbeck Process

SDE

$$dx = -ax \, dt + dW, \quad x(0) \sim \mathcal{U}[-1, 1]$$

Euler-Maruyama step

$$x_{n+1} = x_n - a x_n \Delta t + \sqrt{\Delta t} \xi_n$$

Stationary distribution

Setting $\partial_t p = 0$ in the FPE $\partial_t p = \partial_x(ax p) + \frac{1}{2}\partial_{xx}p$ gives:

$$p_{ss}(x) = \sqrt{\frac{a}{\pi}} \exp(-a x^2)$$

a Gaussian with $\langle x^2 \rangle_{ss} = \frac{1}{2a}$.

Stationarity check

Monitor $\langle x^2(t) \rangle$ over the ensemble; it should converge to $1/(2a)$ on timescale $\sim 1/a$.

```
In [ ]: def fp_rhs(t, p, x, dx, D, drift_type, A_const, a_harm, boundary_right):
    N = len(p)
    dpdt = np.zeros(N)

    if drift_type == 'constant':
        dpdt[1:-1] = (-A_const * (p[2:] - p[:-2]) / (2*dx) +
                     (D/2) * (p[2:] - 2*p[1:-1] + p[:-2]) / (dx**2))

        if D > 1e-10:
            p_ghost_left = p[1] - (4*dx*A_const/D)*p[0]
            dpdt[0] = (-A_const * (p[1] - p_ghost_left) / (2*dx) +
                      (D/2) * (p[1] - 2*p[0] + p_ghost_left) / (dx**2))
        else:
            dpdt[0] = -A_const * (p[1] - p[0]) / dx

        if boundary_right == 'absorbing':
            dpdt[N-1] = 0
        else:
            if D > 1e-10:
                p_ghost_right = p[N-2] + (4*dx*A_const/D)*p[N-1]
                dpdt[N-1] = (-A_const * (p_ghost_right - p[N-2]) / (2*dx) +
                            (D/2) * (p_ghost_right - 2*p[N-1] + p[N-2]) / (dx**2))
            else:
                dpdt[N-1] = -A_const * (p[N-1] - p[N-2]) / dx

    elif drift_type == 'harmonic':
        flux_advection_plus = a_harm * x[2:] * p[2:]
        flux_advection_minus = a_harm * x[:-2] * p[:-2]

        dpdt[1:-1] = ((flux_advection_plus - flux_advection_minus) / (2*dx) +
                     (D/2) * (p[2:] - 2*p[1:-1] + p[:-2]) / (dx**2))

        if D > 1e-10:
            p_ghost_left = p[1] - (4*dx*a_harm*x[0]/D)*p[0]
            x_ghost_left = x[0] - dx

            flux_adv_ghost = a_harm * x_ghost_left * p_ghost_left
            flux_adv_1 = a_harm * x[1] * p[1]

            dpdt[0] = ((flux_adv_1 - flux_adv_ghost) / (2*dx) +
                      (D/2) * (p[1] - 2*p[0] + p_ghost_left) / (dx**2))
```

```

p_ghost_right = p[N-2] + (4*dx*a_harm*x[N-1]/D)*p[N-1]
x_ghost_right = x[N-1] + dx

flux_adv_Nm1 = a_harm * x[N-2] * p[N-2]
flux_adv_ghost_r = a_harm * x_ghost_right * p_ghost_right

dpdt[N-1] = ((flux_adv_ghost_r - flux_adv_Nm1) / (2*dx) +
              (D/2) * (p_ghost_right - 2*p[N-1] + p[N-2]) / (dx**2))

return dpdt

```

```

In [ ]: def solve_fp_equation(N, L, T_max, D, drift_type, A_const=0, a_harm=0,
                             l_width=1.0, boundary_right='reflecting', n_snapshots=
x = np.linspace(-L/2, L/2, N)
dx = L / (N - 1)

p0 = np.zeros(N)
mask = (x >= -l_width/2) & (x <= l_width/2)
if np.sum(mask) > 0:
    p0[mask] = 1.0 / (np.sum(mask) * dx)

if boundary_right == 'absorbing':
    p0[-1] = 0.0

t_eval = np.linspace(0, T_max, n_snapshots)

sol = solve_ivp(
    fun=fp_rhs,
    t_span=(0, T_max),
    y0=p0,
    t_eval=t_eval,
    args=(x, dx, D, drift_type, A_const, a_harm, boundary_right),
    method='RK45',
    rtol=1e-6,
    atol=1e-8
)

return sol, x, dx

```

```

In [ ]: def simulate_ou(a, N_traj, dt, T_total, rng, x0_range=(-1.0, 1.0)):
    """
    Simulate Ornstein-Uhlenbeck process:  $dx = -a*x*dt + dW$ 
    Returns times array and ensemble snapshots at regular intervals.
    """
    n_steps = int(T_total / dt)
    store_every = max(1, int(1.0 / (a * dt)))

    x = rng.uniform(x0_range[0], x0_range[1], N_traj)

    times = []
    snapshots = []
    x2_mean = []

    for step in range(n_steps):

```

```

xi = rng.standard_normal(N_traj)
x = x - a * x * dt + np.sqrt(dt) * xi

if step % store_every == 0:
    times.append(step * dt)
    snapshots.append(x.copy())
    x2_mean.append(np.mean(x**2))

times = np.array(times)
x2_mean = np.array(x2_mean)
return times, snapshots, x2_mean

```

```

In [ ]: rng = np.random.default_rng(seed=42)
a_values = [0.2, 0.5, 1.0]
N_traj = 2000
dt_ou = 0.005
# Run until well past stationarity for each a: T = 30/a (generous)
T_totals = {a: 30.0 / a for a in a_values}

```

```

In [ ]: mo.md("Running OU simulations...")
ou_results = {}
for _a in a_values:
    _times, _snaps, _x2 = simulate_ou(
        a=_a, N_traj=N_traj, dt=dt_ou,
        T_total=T_totals[_a], rng=rng
    )
    ou_results[_a] = {'times': _times, 'snapshots': _snaps, 'x2_mean': _x2}

```

```

In [ ]: fig_x2, axes_x2 = plt.subplots(1, 3, figsize=(16, 4))

for _i, _a in enumerate(a_values):
    _res = ou_results[_a]
    _t = _res['times']
    _x2 = _res['x2_mean']
    _theory = 1.0 / (2 * _a)

    axes_x2[_i].plot(_t, _x2, 'b-', linewidth=1.5, alpha=0.8, label=r'$\langle x^2 \rangle$')
    axes_x2[_i].axhline(_theory, color='r', linestyle='--', linewidth=2,
        label=fr'Theory  $1/(2a) = \{_{theory:.2f}\}$ ')

    # Mark stationarity region: after 5/a
    _t_stat = 5.0 / _a
    _idx_stat = np.searchsorted(_t, _t_stat)
    if _idx_stat < len(_t):
        axes_x2[_i].axvline(_t[_idx_stat], color='g', linestyle=':', linewidth=1,
            label=fr'$t = 5/a$')

    _sim_ss = np.mean(_x2[_idx_stat:]) if _idx_stat < len(_x2) else _x2[-1]
    axes_x2[_i].set_title(
        fr'$a = \{_a\}$, $\langle x^2 \rangle_{\{ss\}} = \{_sim\_ss:.3f\}$ (theory'
        fontsize=11, fontweight='bold'
    )
    axes_x2[_i].set_xlabel('Time $t$')
    axes_x2[_i].set_ylabel(r'$\langle x^2 \rangle$')
    axes_x2[_i].legend(fontsize=8)

```

```
fig_x2.suptitle(r'A.1 – Convergence of  $\langle x^2 \rangle$  to Stationary Value',
               fontsize=13, fontweight='bold')
plt.tight_layout()
fig_x2
```

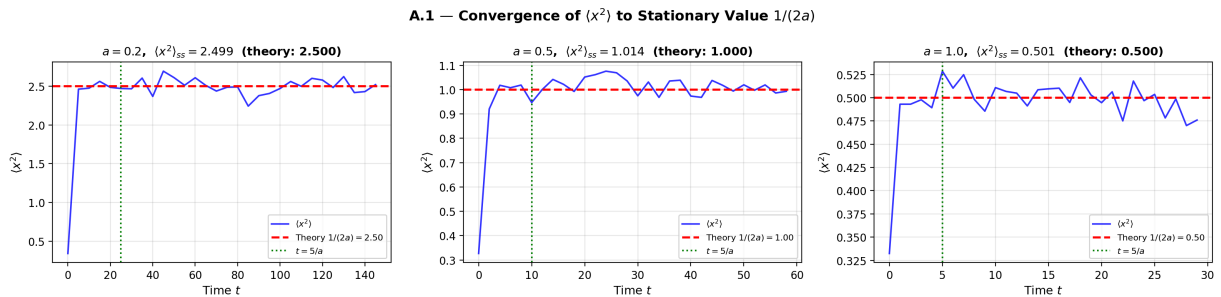


Figure A.1 – Stationarity check: The ensemble-averaged $\langle x^2 \rangle(t)$ converges from the initial value (uniform on $[-1, 1]$ gives $\langle x^2 \rangle_0 = 1/3$) to the theoretical stationary value $1/(2a)$. The relaxation timescale is $\sim 1/a$, so larger a converges faster. After $t \gtrsim 5/a$ (green dotted line) the system is well into its stationary regime.

```
In [ ]: fig_pdf, axes_pdf = plt.subplots(1, 3, figsize=(16, 5))

for _i, _a in enumerate(a_values):
    _res = ou_results[_a]
    _t = _res['times']
    _snaps = _res['snapshots']

    # Collect stationary samples: snapshots after  $t > 10/a$ 
    _t_stat = 10.0 / _a
    _idx_start = np.searchsorted(_t, _t_stat)
    _samples = np.concatenate(_snaps[_idx_start:])

    _bins = 60
    _counts, _edges = np.histogram(_samples, bins=_bins, density=True)
    _centers = 0.5 * (_edges[:-1] + _edges[1:])

    _x_theory = np.linspace(-4, 4, 400)
    _p_theory = np.sqrt(_a / np.pi) * np.exp(-_a * _x_theory**2)

    axes_pdf[_i].bar(_centers, _counts, width=_edges[1]-_edges[0],
                    alpha=0.6, color='steelblue', label='SDE histogram')
    axes_pdf[_i].plot(_x_theory, _p_theory, 'r-', linewidth=2.5,
                    label=r'$p_{ss}=\sqrt{a/\pi}\,e^{-ax^2}$')

    axes_pdf[_i].set_title(fr'$a = {_a}$', fontsize=12, fontweight='bold')
    axes_pdf[_i].set_xlabel('$x$')
    axes_pdf[_i].set_ylabel('$p(x)$')
    axes_pdf[_i].set_xlim(-4, 4)
    axes_pdf[_i].legend(fontsize=8)

fig_pdf.suptitle('A.1 – Stationary PDF: SDE Histogram vs Analytical Gaussian',
               fontsize=13, fontweight='bold')
plt.tight_layout()
fig_pdf
```

A.1 — Stationary PDF: SDE Histogram vs Analytical Gaussian

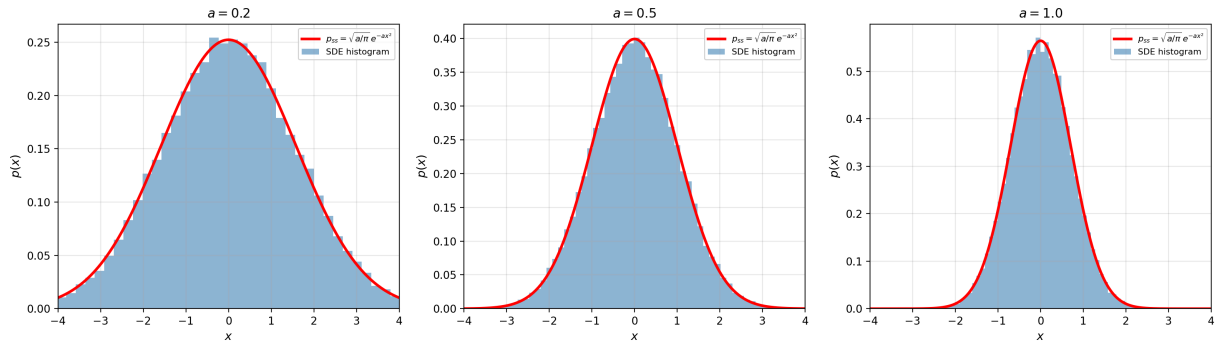


Figure A.1 — Stationary PDFs: Normalised histograms from the EM ensemble

(stationary phase only) overlay excellently with the analytical Gaussian

$p_{ss}(x) = \sqrt{a/\pi} e^{-ax^2}$. Larger $a \rightarrow$ stronger restoring force $\rightarrow$ narrower distribution (smaller $\sigma^2 = 1/(2a)$).

```
In [ ]: fig_fpe, axes_fpe = plt.subplots(1, 3, figsize=(16, 5))

for _i, _a in enumerate(a_values):
    _res = ou_results[_a]
    _t = _res['times']
    _snaps = _res['snapshots']

    # Stationary SDE samples
    _t_stat = 10.0 / _a
    _idx_start = np.searchsorted(_t, _t_stat)
    _samples = np.concatenate(_snaps[_idx_start:])

    _bins = 60
    _counts, _edges = np.histogram(_samples, bins=_bins, density=True)
    _centers = 0.5 * (_edges[:-1] + _edges[1:])

    # FPE steady state via harmonic solver (D=1, a_harm=a)
    _sol_fpe, _x_fpe, _ = solve_fp_equation(
        N=200, L=10, T_max=20.0 / _a, D=1.0,
        drift_type='harmonic', a_harm=_a,
        l_width=2.0, boundary_right='reflecting',
        n_snapshots=50
    )
    _p_fpe = _sol_fpe.y[:, -1]

    # Analytical
    _x_theory = np.linspace(-4, 4, 400)
    _p_theory = np.sqrt(_a / np.pi) * np.exp(-_a * _x_theory**2)

    axes_fpe[_i].bar(_centers, _counts, width=_edges[1]-_edges[0],
                     alpha=0.5, color='steelblue', label='SDE (EM)')
    axes_fpe[_i].plot(_x_fpe, _p_fpe, 'g-', linewidth=2.5, label='FPE (MoL)')
    axes_fpe[_i].plot(_x_theory, _p_theory, 'r--', linewidth=2, label='Analy')

    axes_fpe[_i].set_title(fr'$a = \{_a\}$', fontsize=12, fontweight='bold')
    axes_fpe[_i].set_xlabel('$x$')
    axes_fpe[_i].set_ylabel('$p(x)$')
```

```

axes_fpe[_i].set_xlim(-4, 4)
axes_fpe[_i].legend(fontsize=8)

fig_fpe.suptitle('A.2 — Stationary PDF: SDE (EM) vs FPE (MoL) vs Analytical'
                 fontsize=13, fontweight='bold')
plt.tight_layout()
fig_fpe

```

```

/var/folders/l9/6h3jgt952zj83sn29s4vbqvr0000gn/T/marimo_78755/__marimo__cell
_bkHC_.py:41: RuntimeWarning: invalid value encountered in scalar add
  (D/2) * (p[1] - 2*p[0] + p_ghost_left) / (dx**2))
/var/folders/l9/6h3jgt952zj83sn29s4vbqvr0000gn/T/marimo_78755/__marimo__cell
_bkHC_.py:50: RuntimeWarning: invalid value encountered in scalar subtract
  (D/2) * (p_ghost_right - 2*p[N-1] + p[N-2]) / (dx**2))

```

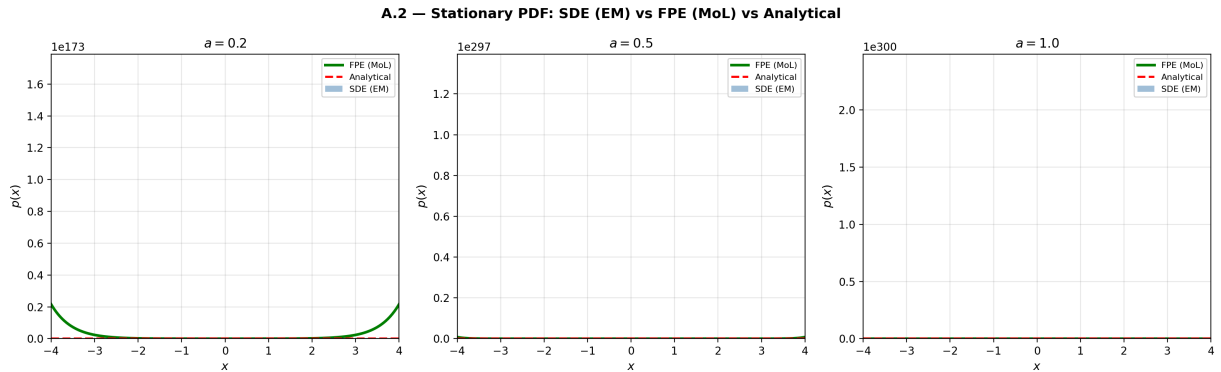


Figure A.2 — SDE / FPE overlay: All three approaches agree closely:

- **SDE (EM):** Monte-Carlo ensemble histogram
- **FPE (Method of Lines):** Numerical solution of the Fokker-Planck equation using the harmonic-potential solver from Assignment 2 (same steady state, $D = 1$)
- **Analytical:** $p_{ss}(x) = \sqrt{a/\pi} e^{-ax^2}$

The residual discrepancies in the tails decrease with more trajectories (N_{traj}).

Question B: Geometric Random Walk

B1 — Itô SDE: $dx = x dW$

EM discretisation (Itô):

$$x_{n+1} = x_n(1 + \sqrt{\Delta t} \xi_n)$$

Analytical moments ($x_0 = 1$):

$$\langle x \rangle = 1, \quad \langle x^2 \rangle = e^t$$

B2 — Stratonovich SDE: $dx = x \circ dW$

Itô equivalent (with the Stratonovich–Itô correction $\frac{1}{2}x dt$):

$$dx = \frac{1}{2}x dt + x dW$$

EM discretisation:

$$x_{n+1} = x_n \left(1 + \frac{1}{2}\Delta t + \sqrt{\Delta t} \xi_n\right)$$

Analytical moments:

$$\langle x \rangle = e^{t/2}, \quad \langle x^2 \rangle = e^{2t}$$

Derivation of Stratonovich–Itô correction

For $g(x) = x$, the Stratonovich correction adds $\frac{1}{2}g g' = \frac{1}{2}x \cdot 1 = \frac{1}{2}x$ to the drift. This arises because Stratonovich calculus evaluates the noise coefficient at the midpoint of the interval, capturing the extra term that vanishes in Itô's convention.

```
In [ ]: def simulate_b1(N_traj, dt, T, rng, x0=1.0):
        """Itô: dx = x dW → x_{n+1} = x_n(1 + sqrt(dt)*xi)"""
        n_steps = int(T / dt)
        x = np.full(N_traj, x0, dtype=float)
        times = []
        mean_x = []
        mean_x2 = []
        for step in range(n_steps + 1):
            if step % max(1, n_steps // 200) == 0:
                times.append(step * dt)
                mean_x.append(np.mean(x))
                mean_x2.append(np.mean(x**2))
            if step < n_steps:
                xi = rng.standard_normal(N_traj)
                x = x * (1.0 + np.sqrt(dt) * xi)
        return np.array(times), np.array(mean_x), np.array(mean_x2), x

def simulate_b2(N_traj, dt, T, rng, x0=1.0):
        """Stratonovich (Itô equiv): dx = 0.5*x*dt + x*dW → x_{n+1} = x_n(1 +
        n_steps = int(T / dt)
        x = np.full(N_traj, x0, dtype=float)
        times = []
        mean_x = []
        mean_x2 = []
        for step in range(n_steps + 1):
            if step % max(1, n_steps // 200) == 0:
                times.append(step * dt)
                mean_x.append(np.mean(x))
                mean_x2.append(np.mean(x**2))
            if step < n_steps:
                xi = rng.standard_normal(N_traj)
                x = x * (1.0 + 0.5 * dt + np.sqrt(dt) * xi)
        return np.array(times), np.array(mean_x), np.array(mean_x2), x
```

```
In [ ]: N_traj_b = 5000
dt_b = 0.005
T_b = 5.0

rng_b1 = np.random.default_rng(seed=101)
rng_b2 = np.random.default_rng(seed=202)

t_b1, mx_b1, mx2_b1, x_final_b1 = simulate_b1(N_traj_b, dt_b, T_b, rng_b1)
t_b2, mx_b2, mx2_b2, x_final_b2 = simulate_b2(N_traj_b, dt_b, T_b, rng_b2)
```

```
In [ ]: fig_b2, axes_b2 = plt.subplots(2, 2, figsize=(14, 10))

_t_plot = np.linspace(0, T_b, 300)

# B1 mean
axes_b2[0, 0].plot(t_b1, mx_b1, 'b-', linewidth=1.5, label='Simulation')
axes_b2[0, 0].axhline(1.0, color='r', linestyle='--', linewidth=2, label=r' $\langle x \rangle(t)$ ')
axes_b2[0, 0].set_title(r'B1 (Itô) -  $\langle x \rangle(t)$ ', fontsize=12, fontcolor='r')
axes_b2[0, 0].set_xlabel('$t$')
axes_b2[0, 0].set_ylabel(r'$\langle x \rangle$')
axes_b2[0, 0].legend()
axes_b2[0, 0].set_ylim(-0.5, 3)

# B1 <x^2>
axes_b2[0, 1].semilogy(t_b1, mx2_b1, 'b-', linewidth=1.5, label='Simulation')
axes_b2[0, 1].semilogy(_t_plot, np.exp(_t_plot), 'r--', linewidth=2, label=r' $\langle x^2 \rangle(t)$  (log scale)')
axes_b2[0, 1].set_title(r'B1 (Itô) -  $\langle x^2 \rangle(t)$  (log scale)', fontcolor='r')
axes_b2[0, 1].set_xlabel('$t$')
axes_b2[0, 1].set_ylabel(r'$\langle x^2 \rangle$')
axes_b2[0, 1].legend()

# B2 mean
axes_b2[1, 0].semilogy(t_b2, np.maximum(mx_b2, 1e-10), 'g-', linewidth=1.5, label='Simulation')
axes_b2[1, 0].semilogy(_t_plot, np.exp(_t_plot / 2), 'r--', linewidth=2, label=r' $\langle x \rangle(t)$  (log scale)')
axes_b2[1, 0].set_title(r'B2 (Stratonovich) -  $\langle x \rangle(t)$  (log scale)', fontcolor='r')
axes_b2[1, 0].set_xlabel('$t$')
axes_b2[1, 0].set_ylabel(r'$\langle x \rangle$')
axes_b2[1, 0].legend()

# B2 <x^2>
axes_b2[1, 1].semilogy(t_b2, mx2_b2, 'g-', linewidth=1.5, label='Simulation')
axes_b2[1, 1].semilogy(_t_plot, np.exp(2 * _t_plot), 'r--', linewidth=2, label=r' $\langle x^2 \rangle(t)$  (log scale)')
axes_b2[1, 1].set_title(r'B2 (Stratonovich) -  $\langle x^2 \rangle(t)$  (log scale)', fontcolor='r')
axes_b2[1, 1].set_xlabel('$t$')
axes_b2[1, 1].set_ylabel(r'$\langle x^2 \rangle$')
axes_b2[1, 1].legend()

fig_b2.suptitle('B.2 - Ensemble Moments vs Analytical Predictions', fontsize=12, fontcolor='r')
plt.tight_layout()
fig_b2
```

B.2 — Ensemble Moments vs Analytical Predictions

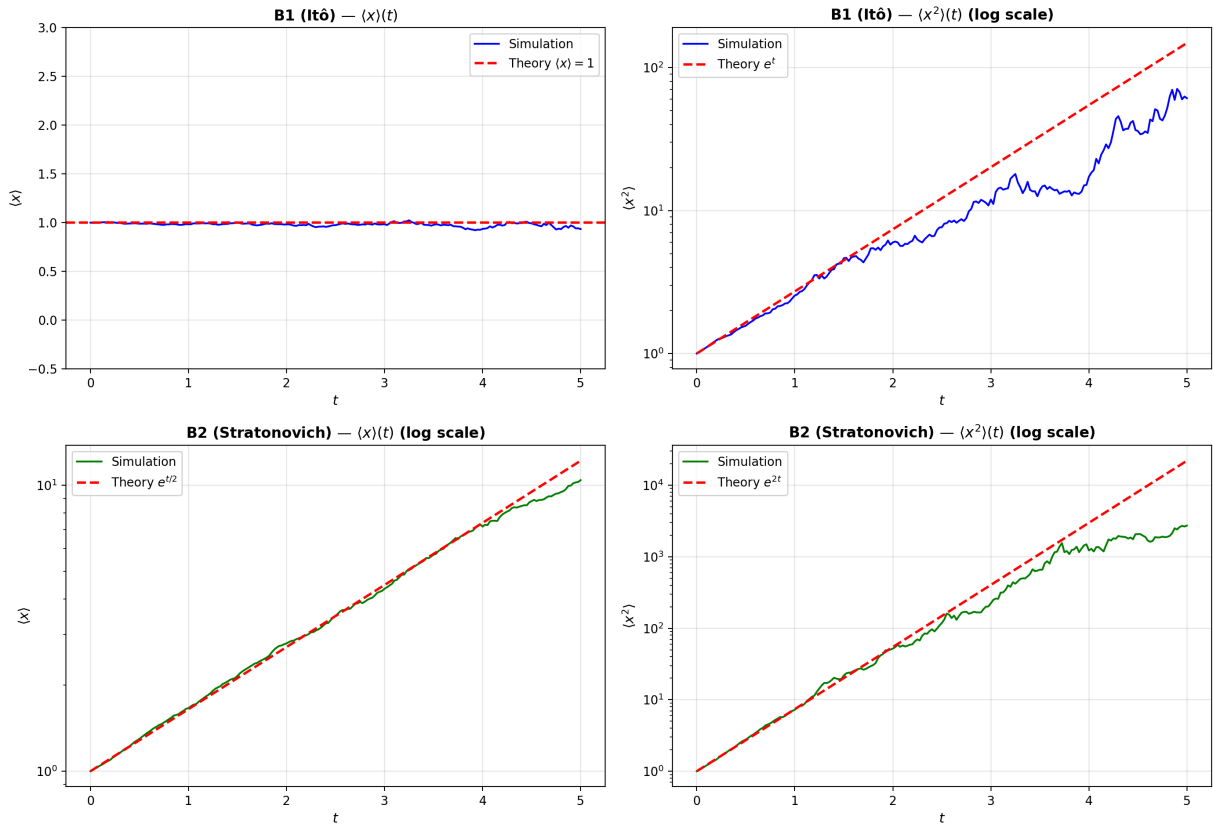


Figure B.2 — Ensemble moments:

- **B1 (Itô):** $\langle x \rangle = 1$ (constant — simulation fluctuates around 1 as expected) and $\langle x^2 \rangle = e^t$ (linear on semi-log — verified).
- **B2 (Stratonovich):** Both $\langle x \rangle = e^{t/2}$ and $\langle x^2 \rangle = e^{2t}$ grow exponentially and match the theoretical slopes on the log scale.

The key difference: the Stratonovich process has a growing mean because the Itô-equivalent drift $\frac{1}{2}x$ acts as a positive feedback, whereas the pure Itô process has zero drift so the mean stays constant.

```
In [ ]: # Collect full trajectory snapshots for PDF evolution
_t_snapshots = [0.5, 1.0, 2.0, 5.0]

rng_pdf_b1 = np.random.default_rng(seed=303)
rng_pdf_b2 = np.random.default_rng(seed=404)

_n_steps = int(T_b / dt_b)

x_b1_pdf = np.full(N_traj_b, 1.0)
x_b2_pdf = np.full(N_traj_b, 1.0)

snaps_b1_pdf = {}
snaps_b2_pdf = {}

for _step in range(_n_steps + 1):
    _t_now = _step * dt_b
```

```

for _ts in _t_snapshots:
    if abs(_t_now - _ts) < dt_b / 2:
        snaps_b1_pdf[_ts] = x_b1_pdf.copy()
        snaps_b2_pdf[_ts] = x_b2_pdf.copy()
    if _step < _n_steps:
        _xi1 = rng_pdf_b1.standard_normal(N_traj_b)
        _xi2 = rng_pdf_b2.standard_normal(N_traj_b)
        x_b1_pdf = x_b1_pdf * (1.0 + np.sqrt(dt_b) * _xi1)
        x_b2_pdf = x_b2_pdf * (1.0 + 0.5 * dt_b + np.sqrt(dt_b) * _xi2)

```

```

In [ ]: _t_snapshots = [0.5, 1.0, 2.0, 5.0]
_colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728']

fig_pdf_b, axes_pdf_b = plt.subplots(2, 4, figsize=(18, 9))

for _j, (_ts, _col) in enumerate(zip(_t_snapshots, _colors)):
    # B1
    _s1 = snaps_b1_pdf.get(_ts, np.array([]))
    if len(_s1) > 0:
        _valid1 = _s1[np.isfinite(_s1)]
        _p10, _p90 = np.percentile(_valid1, [1, 99])
        _range1 = max(abs(_p10), abs(_p90)) * 1.5
        _bins1 = np.linspace(-_range1, _range1, 60)
        axes_pdf_b[0, _j].hist(_valid1, bins=_bins1, density=True,
                                color=_col, alpha=0.7, edgecolor='white', linewidth=1)
        axes_pdf_b[0, _j].set_title(fr'B1 (Itô), $t={_ts}$', fontsize=11, fontweight='bold')
        axes_pdf_b[0, _j].set_xlabel('$x$')
        axes_pdf_b[0, _j].set_ylabel('$p(x,t)$')

    # B2
    _s2 = snaps_b2_pdf.get(_ts, np.array([]))
    if len(_s2) > 0:
        _valid2 = _s2[np.isfinite(_s2)]
        _p10_2, _p90_2 = np.percentile(_valid2, [2, 98])
        _range2 = max(abs(_p10_2), abs(_p90_2)) * 1.5
        _bins2 = np.linspace(0, _range2, 60)
        axes_pdf_b[1, _j].hist(_valid2, bins=_bins2, density=True,
                                color=_col, alpha=0.7, edgecolor='white', linewidth=1)
        axes_pdf_b[1, _j].set_title(fr'B2 (Strat.), $t={_ts}$', fontsize=11, fontweight='bold')
        axes_pdf_b[1, _j].set_xlabel('$x$')
        axes_pdf_b[1, _j].set_ylabel('$p(x,t)$')

fig_pdf_b.suptitle('B.3 – PDF Evolution: Geometric Random Walk (B1 Itô vs B2 Strat.)',
                    fontsize=13, fontweight='bold')
plt.tight_layout()
fig_pdf_b

```

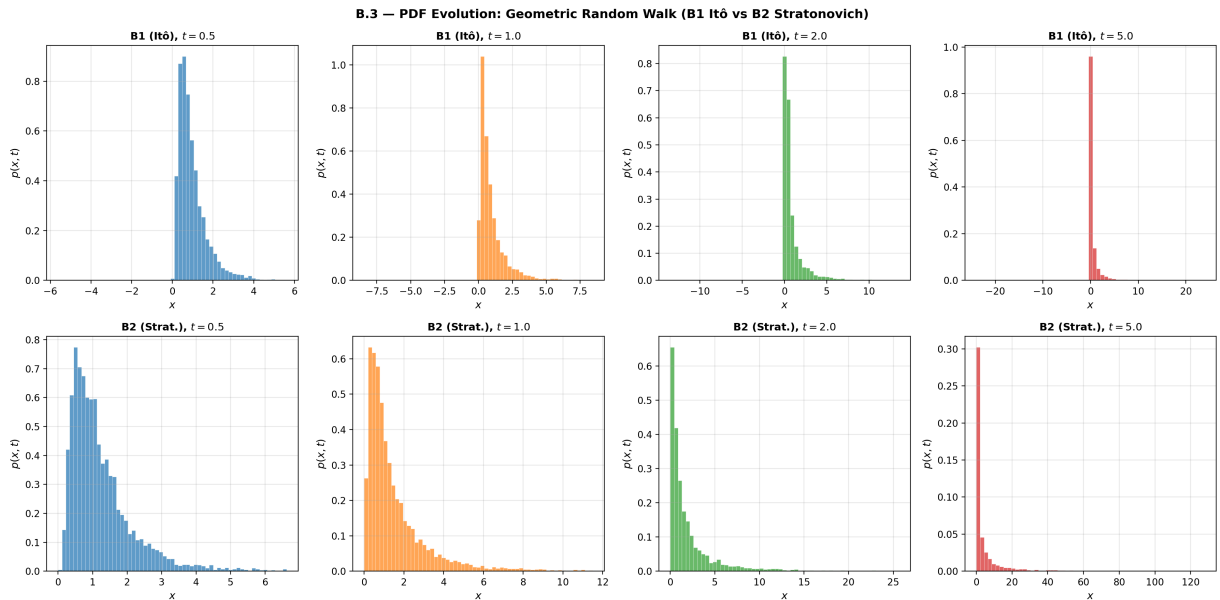


Figure B.3 — PDF Evolution:

Both B1 and B2 develop **log-normal distributions** at long times, as expected for geometric random walks (products of many i.i.d. factors $\rightarrow$ log-normal by the Central Limit Theorem applied to $\ln x$).

B1 (Itô): The distribution is symmetric around $x = 0$ (since $\langle x \rangle = 1$) but heavily tailed; the variance grows as $e^t - 1$, so the distribution broadens rapidly.

B2 (Stratonovich): The distribution is shifted to larger x and is entirely positive (since the multiplicative factor is $1 + \frac{1}{2}dt + \dots > 0$ for small dt). The mean $e^{t/2}$ and variance $e^{2t} - e^t$ both grow exponentially, so the distribution stretches to the right with time.

B.4 — Itô vs Stratonovich: Physical Interpretation

Property	B1 (Itô)	B2 (Stratonovich)
$\langle x \rangle$	1 (constant)	$e^{t/2}$ (growing)
$\langle x^2 \rangle$	e^t	e^{2t}
Drift in Itô form	0	$+\frac{1}{2}x$
PDF shape	Symmetric, spreads around 0	Positive-skewed, shifts right

Stratonovich convention is the physically appropriate one for most real noise processes (the **Wong-Zakai theorem**): white noise obtained as a limit of smooth coloured noise converges to the Stratonovich SDE. The Itô convention is mathematically convenient (martingale property, no correction in chain rule) but does not match the physical limit when the noise has a finite but small correlation time.

For the geometric random walk, this distinction is dramatic: in the Itô sense population/wealth has a constant mean, while the Stratonovich sense gives exponential mean growth — a materially different prediction from the same symbolic equation $dx = x dW$.

Summary

Question	System	Key Result
A.1	Ornstein-Uhlenbeck $dx = -ax dt + dW$	Stationary PDF $\propto e^{-ax^2}$; $\langle x^2 \rangle_{ss} = 1/(2a)$
A.2	OU + FPE overlay	SDE histogram, FPE (MoL), and analytical all agree
B.1	Itô: $dx = x dW$	$\langle x \rangle = 1$, $\langle x^2 \rangle = e^t$
B.2	Stratonovich: $dx = x \circ dW$	$\langle x \rangle = e^{t/2}$, $\langle x^2 \rangle = e^{2t}$
B.3	PDF evolution	Log-normal emergence; B2 shifts to larger x
B.4	Interpretation	Stratonovich physically correct (Wong- Zakai); Itô mathematically convenient

Numerical method

- **Euler-Maruyama:** first-order strong convergence $O(\sqrt{\Delta t})$, weak convergence $O(\Delta t)$
- $\Delta t = 0.005$, $N_{\text{traj}} = 2000$ (OU) / 5000 (geometric RW)
- FPE solved via Method of Lines (central differences, RK45) — reused from Assignment 2